EXPERIMENT 10

Write a Python program to implement the A* algorithm

Register Number: **192511013**

192511013
192511013
192511013

Aim

To write a Python program to implement the A* search algorithm, which finds the least-cost path from a start node to a goal node using the evaluation function f(n) = g(n) + h(n).

Algorithm

Step 1: Represent the weighted graph as an adjacency list with edge costs, and define the heuristic value h(n) (estimated cost from n to the goal) for every node.

Step 2: Create an open list (priority queue) ordered by f(n) = g(n) + h(n), and insert the start node with g = 0. Create an empty closed set.

Step 3: Remove the node with the lowest f value from the open list. If it is already in the closed set, skip it.

Step 4: If the removed node is the goal, trace the parent links back to the start to obtain the path, and stop. The value g(goal) is the path cost.

Step 5: Otherwise add the node to the closed set and examine each neighbour: compute new g = g(node) + edge cost.

Step 6: If the neighbour has not been reached before or the new g is smaller than its recorded g, update its g and parent, and insert it into the open list with f = g + h.

Step 7: Repeat from step 3 until the goal is found or the open list becomes empty (no path). Print the nodes expanded, the path and its cost.

192511013

Program

```
import heapq

# graph[node] = list of (neighbour, edge cost)
graph = {
    'S': [('A', 1), ('B', 4)],
    'A': [('B', 2), ('C', 5), ('D', 12)],
    'B': [('C', 2)],
    'C': [('D', 3), ('G', 7)],
    'D': [('G', 2)],
    'G': []
}

# heuristic h(n): estimated cost from n to the goal G (never over-estimates)
h = {'S': 7, 'A': 6, 'B': 4, 'C': 5, 'D': 2, 'G': 0}

def a_star(start, goal):
    open_list = [(h[start], 0, start)]          # (f, g, node)
    parent = {start: None}
    g_cost = {start: 0}
    closed = set()
    step = 1
    while open_list:
        f, g, node = heapq.heappop(open_list)
        if node in closed:
            continue
        print(f"Step {step}: expand {node}  (g = {g}, h = {h[node]}, f = {f})")
        step += 1
        if node == goal:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return path[::-1], g
        closed.add(node)
        for neighbour, cost in graph[node]:
            new_g = g + cost
            if neighbour not in g_cost or new_g < g_cost[neighbour]:
                g_cost[neighbour] = new_g
                parent[neighbour] = node
                heapq.heappush(open_list, (new_g + h[neighbour], new_g, neighbour))
    return None, float('inf')

if __name__ == "__main__":
    path, cost = a_star('S', 'G')
    if path:
        print("\nShortest path:", ' -> '.join(path))
        print("Total cost   :", cost)
    else:
        print("No path found")
```

192511013

Output

```
Step 1: expand S  (g = 0, h = 7, f = 7)
Step 2: expand A  (g = 1, h = 6, f = 7)
Step 3: expand B  (g = 3, h = 4, f = 7)
Step 4: expand C  (g = 5, h = 5, f = 10)
Step 5: expand D  (g = 8, h = 2, f = 10)
Step 6: expand G  (g = 10, h = 0, f = 10)

Shortest path: S -> A -> B -> C -> D -> G
Total cost   : 10
```

192511013